

Absolutely brooo 🔥. Let's make this your **permanent JavaScript Async + API revision notes** — something you can come back to before interviews/projects.

I'll keep it **concise but with enough explanation**, and include the important code patterns we actually used.

JavaScript Async + Fetch API — Revision Notes

1. Synchronous JavaScript

Definition

Synchronous JavaScript executes code **one statement at a time, in order**.

The next statement waits until the current statement finishes.

```
js

console.log("Start");

console.log("Middle");

console.log("End");
```

Output:

```
text

Start
Middle
End
```

Mental model

```
text

Task 1
↓
Task 2
↓
Task 3
↓
Task 4
```

Problem

If one operation takes a long time:

```
js

console.log("Start");

for (let i = 0; i < 10000000000; i++) {
```

```
// huge work  
}  
  
console.log("End");
```

JavaScript cannot reach "End" until the loop finishes.

2. Asynchronous JavaScript

Definition

Asynchronous JavaScript allows JavaScript to start a time-consuming operation and continue with other work instead of waiting for that operation to finish.

Common asynchronous operations:

- Timers
- Network/API requests
- File operations
- Database operations in server-side JS

Example

```
js  
  
console.log("Start");  
  
setTimeout(() => {  
    console.log("Middle");  
}, 2000);  
  
console.log("End");
```

Output:

```
text  
  
Start  
End  
Middle
```

Why?

```
text  
  
Start  
↓  
setTimeout starts  
↓  
Don't wait  
↓  
End  
↓  
2 seconds later  
↓
```

Important

`setTimeout(..., 2000)` means the callback becomes eligible **after roughly 2 seconds**. It does not guarantee execution at exactly 2 seconds.

3. Callback

Definition

A **callback** is a function passed to another function so that it can be called later or when some operation is completed.

Example:

```
js

function greet() {
  console.log("Hello bro!");
}

function executeSomething(callback) {
  callback();
}

executeSomething(greet);
```

Here:

```
js

executeSomething(greet);
```

passes `greet` to `executeSomething` .

Then:

```
js

callback();
```

executes it.

⚠ Function vs Function Call

Pass the function

```
js

executeSomething(greet);
```

Execute immediately

```
js

executeSomething(greet());
```

These are different.

```
text

greet
↓
function itself

greet()
↓
execute the function now
```

4. Callback with Asynchronous Code

```
js

function downloadFile(callback) {

  console.log("Downloading...");

  setTimeout(() => {

    console.log("Download complete!");

    callback();

  }, 3000);

}

downloadFile(() => {
  console.log("Open the file!");
});
```

Flow:

```
text

downloadFile()
↓
Downloading...
↓
wait
↓
Download complete!
↓
callback()
↓
Open the file!
```

Mental model

Callback = "When you're done, execute this function."

5. Callback Hell

Definition

Callback Hell happens when many asynchronous operations depend on one another and callbacks become deeply nested.

Example:

```
js

getStudent(() => {

  getCourses(() => {

    getAssignments(() => {

      getResults(() => {

        console.log("Everything done");

      });

    });

  });

});
```

Why is it bad?

It becomes:

- Hard to read
- Hard to debug
- Hard to modify
- Difficult to handle errors

Often called the:

Pyramid of Doom

Problem

```
text

Operation 1
  ↓
  callback
  ↓
Operation 2
  ↓
  callback
  ↓
Operation 3
  ↓
  callback
  ↓
Operation 4
```

Solution

6. Promise

Definition

A **Promise** is an object representing the eventual result of an asynchronous operation.

It can be:

text

-  Pending
-  Fulfilled
-  Rejected

Example:

js

```
const promise = new Promise((resolve, reject) => {  
    resolve("Success!");  
});
```

Promise States

Pending

Operation is still running.

text

-  Pending

Fulfilled

Operation completed successfully.

js

```
resolve(data);
```

text

-  Fulfilled

Rejected

Operation failed.

js

```
reject(error);
```

text

✖ Rejected

7. resolve() and reject()

Inside a Promise:

```
js
new Promise((resolve, reject) => {
});
```

resolve()

Used when operation succeeds.

```
js
resolve("Student received");
```

reject()

Used when operation fails.

```
js
reject("Server error");
```

Mental model:

```
text
resolve() → SUCCESS ✔
reject() → FAILURE ✖
```

8. Asynchronous Promise Example

```
js
function getStudent() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve("Student data received");
    }, 2000);
  });
}
```

```
}
```

When calling:

```
js
```

```
const result = getStudent();
```

`result` is initially a:

```
text
```

```
Promise { <pending> }
```

It is **not immediately the student data**.

After 2 seconds:

```
text
```

```
Promise  
  ↓  
resolve()  
  ↓  
Fulfilled
```

9. `.then()`

Definition

`.then()` is used to handle the **successful result of a Promise**.

```
js
```

```
getStudent()  
  .then((student) => {  
    console.log(student);  
  });
```

If we have:

```
js
```

```
resolve("Vamsi");
```

then:

```
js
```

```
.then((student) => {  
  console.log(student);  
});
```

receives:

```
text
```

```
student = "Vamsi"
```

Mental model:

```
text

resolve(data)
  ↓
.then(data)
```

10. .catch()

Definition

`.catch()` is used to handle a **rejected Promise/error**.

```
js

getStudent()
  .then((student) => {
    console.log(student);
  })
  .catch((error) => {
    console.log(error);
  });
```

Mental model:

```
text

resolve()
  ↓
.then()

reject()
  ↓
.catch()
```

11. Promise Chaining

Promises allow multiple asynchronous operations to be chained.

```
js

getStudent()
  .then((student) => {

    return getCourses(student);

  })
  .then((courses) => {

    return getAssignments(courses);

  })
```

```

.then((assignments) => {

    console.log(assignments);

})
.catch((error) => {

    console.log(error);

});

```

Flow:

```

text

getStudent()
  ↓
student
  ↓
getCourses(student)
  ↓
courses
  ↓
getAssignments(courses)
  ↓
assignments

```

★ Why return ?

This is VERY important.

```

js

.then((student) => {

    return getCourses(student);

})

```

We return the Promise from `getCourses()` so the next `.then()` waits for it and receives its result.

Without `return` :

```

js

.then((student) => {

    getCourses(student);

})

```

the Promise chain is not connected to `getCourses()` .

Remember:

Return the next Promise when chaining asynchronous operations.

12. `async`

Definition

`async` is used to declare an **asynchronous function**.

```
js

async function loadData() {

}
```

An `async` function **always returns a Promise**.

Example:

```
js

async function hello() {

    return "Hello";

}
```

Even though we return a normal string, calling:

```
js

hello();
```

produces a Promise.

13. `await`

Definition

`await` waits for a Promise's result **before continuing that async function**.

```
js

async function loadStudent() {

    const student = await getStudent();

    console.log(student);

}
```

Mental model:

```
text

getStudent()
  ↓
Promise
  ↓
await
```

```
↓  
wait for result  
↓  
student
```

`.then()` vs `await`

Promise style

```
js  
  
getStudent()  
  .then((student) => {  
    console.log(student);  
  });
```

Async/await style

```
js  
  
async function loadStudent() {  
  
  const student = await getStudent();  
  
  console.log(student);  
  
}
```

`async/await` usually makes asynchronous code easier to read.

14. Important `await` concept

This:

```
js  
  
const student = await getStudent();
```

does **NOT** mean:

"Freeze the entire browser."

It means:

Pause this async function's continuation until the Promise settles.

Other JavaScript/browser work can continue.

15. `try / catch`

Definition

`try/catch` is used to **handle errors without letting them break normal program execution unexpectedly.**

Basic syntax:

```
js

try {

    // code that might cause an error

} catch (error) {

    // handle error

}
```

Example:

```
js

try {

    throw new Error("Something went wrong");

} catch (error) {

    console.log(error);

}
```

16. async/await + try/catch

This is a very common real-world pattern:

```
js

async function loadStudent() {

    try {

        const student = await getStudent();

        console.log(student);

    } catch (error) {

        console.log(error);

    }

}
```

Mental model:

```
text

async function
  ↓
await Promise
  ↓
Success → continue
```

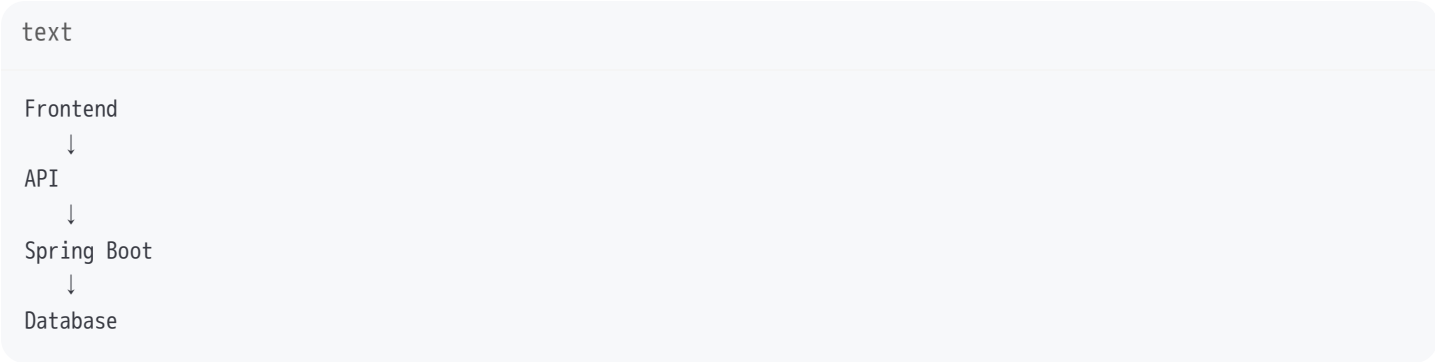
Error → catch

17. API

Definition

An **API** is a way for applications/software components to communicate with each other.

In our project:



Your JavaScript can ask Spring Boot:

"Give me all students."

or:

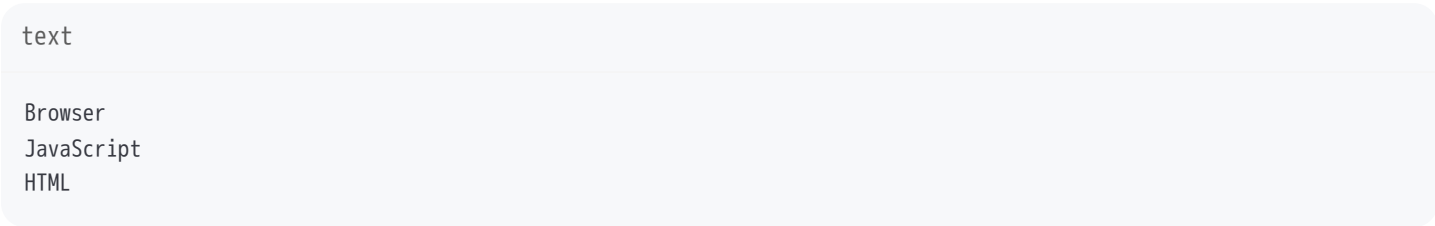
"Add this student."

18. Client and Server

Client

The application making the request.

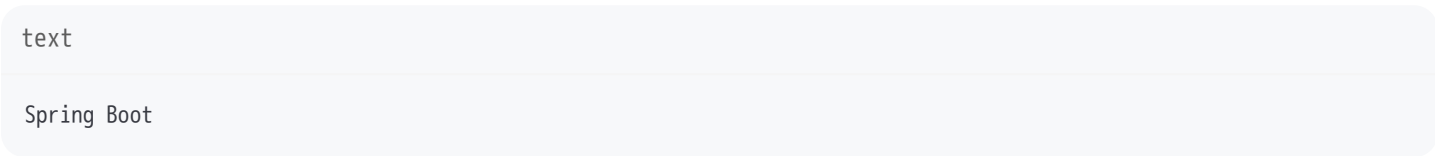
For us:



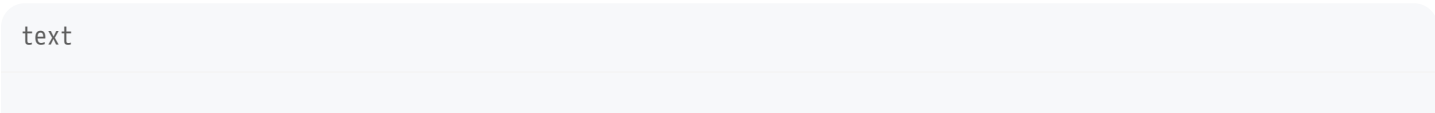
Server

The application receiving and processing requests.

For us:



Flow:





19. HTTP Methods

The most important CRUD methods:

Method	Purpose
GET	Read/get data
POST	Create data
PUT	Update data
DELETE	Delete data

Remember:

```
text

GET    → Give me data
POST   → Create data
PUT    → Update data
DELETE → Delete data
```

20. JSON

Definition

JSON (JavaScript Object Notation) is a text-based format commonly used to exchange data between applications.

Example:

```
json

{
  "name": "Vamsi",
  "age": 20,
  "course": "CSE"
}
```

JavaScript Object → JSON

Use:

```
js
```

```
JSON.stringify(student);
```

Example:

```
js

const student = {
  name: "Vamsi",
  age: 20
};

const jsonData = JSON.stringify(student);
```

JSON → JavaScript Object

Use:

```
js

JSON.parse(jsonData);
```

Mental model:

```
text

JSON.stringify()
Object → JSON string

JSON.parse()
JSON string → JavaScript value
```

21. Fetch API

Definition

`fetch()` is a **built-in JavaScript API** used to make HTTP requests.

It returns a **Promise**.

Basic:

```
js

fetch(url);
```

Because it returns a Promise:

```
js

const response = await fetch(url);
```

22. GET with Fetch

Example:

```
js

async function getUsers() {

  try {

    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );

    const users = await response.json();

    console.log(users);

  } catch (error) {

    console.log(error);

  }

}
```

23. Why TWO `await` s?

This is extremely important.

```
js

const response = await fetch(url);

const data = await response.json();
```

They perform different jobs.

First

```
js

await fetch(url);
```

Waits for the HTTP response.

Second

```
js

await response.json();
```

Reads/parses the response body as JSON.

Flow:

```
text
```

```
fetch()
↓
HTTP Response
↓
response
↓
response.json()
↓
JavaScript data
```

24. response

`fetch()` gives us a **Response object**, not directly the JSON data.

```
js

const response = await fetch(url);
```

The response contains things such as:

```
text

status
headers
body
ok
```

Then:

```
js

const data = await response.json();
```

gives us the actual JavaScript data.

25. response.ok

HTTP errors like `404` or `500` need special attention with Fetch.

A common pattern:

```
js

if (!response.ok) {
  throw new Error("Request failed");
}
```

Complete:

```
js

try {

  const response = await fetch(url);
```

```

    if (!response.ok) {
      throw new Error("Request failed");
    }

    const data = await response.json();
  } catch (error) {

    console.log(error);
  }

```

26. Fetch + DOM

Suppose HTML has:

```

html

<div id="students"></div>

```

Fetch gives us:

```

js

const students = await response.json();

```

Now we can display them.

```

js

const container = document.getElementById("students");

students.forEach((student) => {

  const div = document.createElement("div");

  div.textContent =
    `${student.id} - ${student.name} - ${student.age}`;

  container.appendChild(div);

});

```

What's happening?

```

text

API
↓
JSON
↓
JavaScript array
↓
forEach()
↓
createElement()

```

↓
textContent
↓
appendChild()
↓
DOM changes
↓
Browser displays data

27. Connecting Fetch to YOUR Spring Boot GET API

Your backend:

```
java

@RestController
@RequestMapping("/students")
@CrossOrigin(origins = "http://127.0.0.1:5500")
public class StudentController {

    @GetMapping("/all")
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }
}
```

Frontend:

```
js

async function getStudents() {

    try {

        const response = await fetch(
            "http://localhost:8080/students/all"
        );

        if (!response.ok) {
            throw new Error("Failed to fetch students");
        }

        const students = await response.json();

        console.log(students);

    } catch (error) {

        console.log(error);

    }

}
```

Flow:

text

```
HTML / JavaScript
↓
fetch()
↓
GET
↓
localhost:8080
↓
Spring Boot
↓
StudentService
↓
Student data
↓
JSON
↓
JavaScript
```

28. Fetch POST

GET:

```
js

fetch(url);
```

POST needs configuration.

```
js

fetch(url, {

  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify(student)

});
```

29. POST – Complete Pattern

Suppose:

```
js

const student = {
  name: "Vamsi",
  age: 20,
  course: "ECE"
};
```

Send it:

js

```
async function addStudent() {

  try {

    const response = await fetch(
      "http://localhost:8080/students/add",
      {
        method: "POST",

        headers: {
          "Content-Type": "application/json"
        },

        body: JSON.stringify(student)
      }
    );

    if (!response.ok) {
      throw new Error("Failed to add student");
    }

    const data = await response.json();

    console.log("Student added:", data);

  } catch (error) {

    console.log(error);

  }

}
```

30. Understanding POST options

This:

js

```
{
  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify(student)
}
```

is the **Fetch configuration/options object**.

method

js

```
method: "POST"
```

Tells Fetch which HTTP method to use.

headers

```
js
```

```
"Content-Type": "application/json"
```

Tells the server:

```
"The data I'm sending is JSON."
```

body

```
js
```

```
body: JSON.stringify(student)
```

Contains the actual data being sent.

31. Spring Boot POST

Frontend sends:

```
json
```

```
{
  "name": "Vamsi",
  "age": 20,
  "course": "ECE"
}
```

Spring Boot:

```
java
```

```
@PostMapping("/add")
public Student addStudent(@RequestBody Student student) {

    return studentService.addstudent(student);

}
```

`@RequestBody` tells Spring:

```
Take the JSON request body and convert it into a Java Student object.
```

Flow:

```
text
```

```
JavaScript Object
↓
```

```
JSON.stringify()
  ↓
JSON
  ↓
HTTP POST
  ↓
Spring Boot
  ↓
@RequestBody
  ↓
Java Student object
```

32. CORS

Definition

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether a frontend from one origin can access resources from another origin.

Our frontend:

```
text

http://127.0.0.1:5500
```

Backend:

```
text

http://localhost:8080
```

These are different origins.

Therefore the browser can block the request unless the backend allows the frontend origin.

33. Simple Spring Boot CORS Fix

Controller:

```
java

@CrossOrigin(origins = "http://127.0.0.1:5500")
```

Example:

```
java

@RestController
@RequestMapping("/students")
@CrossOrigin(origins = "http://127.0.0.1:5500")
public class StudentController {

    @GetMapping("/all")
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }
}
```

```

@PostMapping("/add")
public Student addStudent(@RequestBody Student student) {
    return studentService.addstudent(student);
}
}

```

This tells Spring Boot:

Allow requests from `http://127.0.0.1:5500` .

34. Why POST sometimes causes an OPTIONS request

For some cross-origin requests, especially JSON POST requests, the browser first sends a **preflight request**:

```

text

OPTIONS /students/add

```

The browser is basically asking:

"Am I allowed to make this POST request?"

If Spring Boot doesn't allow it:

```

text

OPTIONS
↓
✗ CORS
↓
POST doesn't happen

```

If allowed:

```

text

OPTIONS
↓
✓ Allowed
↓
POST
↓
Spring Boot

```

This is why you saw the CORS error while GET was working.

35. The Complete Full-Stack Flow You Built

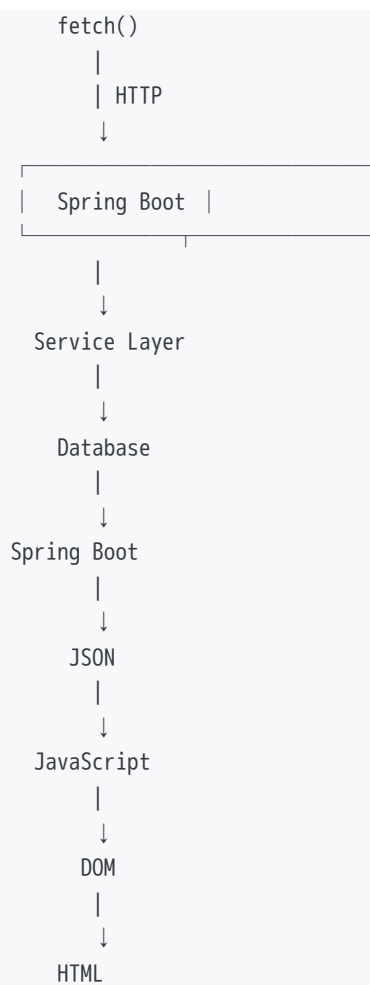
This is the **most important diagram to remember**:

```

text

      BROWSER
HTML + JavaScript
  |
  |

```



★ MASTER CHEAT SHEET

If you forget everything, remember this:

text

Synchronous

→ Do things one by one.

Asynchronous

→ Don't block while waiting for slow operations.

Callback

→ "When you're done, run this function."

Callback Hell

→ Too many nested callbacks.

Promise

→ Represents a future result.

resolve()

→ Promise succeeded.

reject()

→ Promise failed.

.then()

→ Handle Promise success.

`.catch()`
→ Handle Promise failure.

`async`
→ Declares an async function; it returns a Promise.

`await`
→ Wait for a Promise result inside an async function.

`try/catch`
→ Handle errors.

API
→ Communication between applications.

HTTP
→ Protocol used for web communication.

GET
→ Read data.

POST
→ Create/send data.

JSON
→ Common data-exchange format.

`JSON.stringify()`
→ JavaScript value → JSON string.

`JSON.parse()`
→ JSON string → JavaScript value.

`fetch()`
→ Make HTTP requests; returns a Promise.

`response.json()`
→ Read/parse response body as JSON.

CORS
→ Browser security mechanism for cross-origin requests.

`@CrossOrigin`
→ Allows specified frontend origins in Spring Boot.

🔥 The 15-line Fetch pattern to remember

When you eventually forget Fetch syntax, come back to this:

```
js

async function getData() {

    try {

        const response = await fetch(url);

        if (!response.ok) {
            throw new Error("Request failed");
        }
    }
}
```

```

    }

    const data = await response.json();

    console.log(data);

  } catch (error) {

    console.log(error);

  }

}

```

And POST:

```

js

const response = await fetch(url, {

  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify(data)

});

```

Your learning progress

You've now covered:

text

- ✓ Synchronous JS
- ✓ Asynchronous JS
- ✓ setTimeout
- ✓ Callbacks
- ✓ Callback Hell
- ✓ Promises
- ✓ resolve / reject
- ✓ .then()
- ✓ .catch()
- ✓ Promise chaining
- ✓ async
- ✓ await
- ✓ try / catch
- ✓ APIs
- ✓ HTTP basics
- ✓ JSON
- ✓ Fetch
- ✓ GET
- ✓ POST
- ✓ Fetch + DOM
- ✓ Fetch → Spring Boot
- ✓ @RequestBody

- ✓ CORS
- ✓ CORS preflight / OPTIONS

So bro, **you've completed the entire async + basic API foundation.** 🔥

The next major topics in your roadmap are **PUT + DELETE, Axios, Fetch vs Axios, and JavaScript Classes**. Since you said to skip the remaining CRUD practice, we can move straight to **Axios** and then **Classes**.